

Casos de Uso

Iniciar Sesión:

Decidimos no meter el CU "Iniciar Sesión" en el diagrama general para no hacer bulto y que el gráfico se entienda rápido a simple vista. Igual, como el tema de la seguridad y el login es súper importante, lo dejamos anotado como una precondition obligatoria en la documentación detallada de cada caso de uso.

Registrar Donante y Registrar Entidad Beneficiaria:

Preferimos separar estos dos en casos de uso distintos dentro del diagrama en vez de meter todo en un mismo "Registrar Usuario". Básicamente lo hicimos porque los datos que manejamos no tienen nada que ver: a los donantes les validamos el DNI, género o cosas de su empresa, mientras que las entidades son más del tipo escuela o comedor y nos importan sus datos y quiénes son sus representantes.

Exponer Notificación:

Separamos la lógica de los mensajes en el CU "Exponer Notificación", el cual termina siendo incluido por "Importar Donantes", "Registrar Donante" y "Registrar Entidad Beneficiaria". A su vez, lo conectamos con un actor secundario ("Servicio Externo de Notificación") para representar la integración con plataformas de Email, SMS y WhatsApp.

Diagrama de Clases - Donaciones

Donante (Persona Humana y Jurídica)

Modelamos Donante como una clase abstracta que es extendida por PersonaHumana y PersonaJuridica. La razón principal es centralizar los atributos y comportamientos comunes (como el identificador único, la lista de medios de contacto, el contacto predeterminado y el histórico de sus donaciones) en la clase madre. Esto evita la duplicación de código y respeta el Principio de Sustitución de Liskov (LSP), permitiendo que el resto del sistema interactúe con la abstracción Donante de manera uniforme, sin importar si es una persona física o una organización corporativa.

Representante

Decidimos modelar a Representante como una clase independiente asociada de forma exclusiva a una lista dentro de PersonaJuridica. No tenía sentido técnico obligar al representante a heredar de Donante, ya que representa un contacto administrativo o empleado autorizado para operar en nombre de la empresa, mas no es el aportante real de los bienes. De esta forma, aislamos sus datos y evitamos ensuciar el histórico global de donaciones con registros de personal corporativo.

Medio de Contacto y Contacto Predeterminado

Para saber con precisión a dónde redirigir las notificaciones críticas del sistema, incluimos el atributo contactoPredeterminado dentro de la clase Donante, el cual apunta directamente a una instancia elegida dentro de su colección de MedioContacto. El beneficio directo es la

simplicidad del diseño y la alta cohesión: el sistema puede mutar el canal preferido (EMAIL, SMS o WHATSAPP) en tiempo de ejecución cambiando únicamente una referencia, sin alterar las lógicas de negocio asociadas al perfil del usuario.

Bienes, Categorías y Subcategorías

En lugar de fragmentar el modelo en subclases rígidas como "BienUsado" o "BienPerecedero", unificamos las propiedades en una única clase Bien con atributos opcionales (estado, fechaVencimiento). Es la clase Categoria la que asume la responsabilidad de contener las banderas lógicas de configuración (pideEstado, esPerecedero). Con esto evitamos enredos de herencia: si un dato no aplica para una subcategoría específica (por ejemplo, fecha de vencimiento en una silla de madera), simplemente se gestiona como nulo o vacío, permitiendo que las reglas de validación sean dinámicas y escalables.

Modelado de la Segmentación de Donaciones

Introducimos la clase RegistroDonacion para encapsular la carga masiva original de mercadería mezclada que ingresa físicamente al depósito. Para resolver la división automática exigida por el negocio, diseñamos el componente SegmentadorDeDonacion. Su función exclusiva (Principio de Responsabilidad Única - SRP) es procesar el registro original y descomponerlo en múltiples objetos Donacion independientes y atómicos, cada uno estrictamente asociado a una única Subcategoría, garantizando que el motor de emparejamiento reciba unidades mínimas de asignación perfectamente limpias.

Modelado del Estado de la Donación

Para cumplir con la estricta trazabilidad y auditoría requerida, la clase Donacion no sobrescribe su estado actual en una variable simple, sino que mantiene una lista de tipo CambioEstado (historialEstados). Cada cambio de estado almacena su respectiva fecha, el tipo de estado correspondiente y una justificación textual obligatoria (por ejemplo, los motivos de una entrega fallida). Esto asegura poder reconstruir la línea de tiempo completa del ciclo de vida de cualquier bien dentro de la ONG sin pérdida de información histórica.

Satisfacción de Necesidades (TipoNecesidad)

Las necesidades de las entidades beneficiarias evalúan su cumplimiento de formas opuestas: las extraordinarias acumulan cantidades globales fijas ante emergencias y las recurrentes validan objetivos de consumo en base a períodos de tiempo establecidos (semanal, mensual, etc.). Aplicamos el patrón Strategy, encapsulando esta lógica de evaluación polimórfica en la interfaz TipoNecesidad. De este modo, la clase Necesidad delega la comprobación de su estado y cumple con el Principio de Abierto/Cerrado (OCP), quedando totalmente blindada ante la futura incorporación de nuevas reglas de satisfacción.

Asignación de Donaciones (Motor de Matchmaking)

Para el núcleo de emparejamiento de recursos implementamos el patrón Strategy. La clase MotorDeMatchmaking actúa como el contexto que recibe y ejecuta una colección de componentes del tipo AlgoritmoAsignacion. Esto nos permite alternar de forma transparente entre criterios de ordenamiento tan distintos como CompatibilidadSemantica o PrioridadSubAtendidos para consolidar el ResultadoMatchmaking final. Esta solución de

diseño desacopla las reglas de negocio del motor principal, permitiendo agregar o modificar algoritmos de ranking sin alterar una sola línea de código del componente distribuidor.

Arquitectura de Eventos Local (Patrón Observer)

Para evitar que las transacciones del núcleo del negocio se acoplen con lógicas secundarias de envío de alertas o integraciones con APIs externas, implementamos el patrón Observer a través de despachadores de eventos locales independientes en cada contexto (EventManagerDonaciones y EventManagerLogistica). Las clases de negocio simplemente invocan el método emitir(evento) y se desentienden del resultado. Los listeners concretos (como EntregaExitosaListener o InicioRutaListener) capturan la señal de forma asíncrona en segundo plano, garantizando un código limpio con separación total de responsabilidades.

Componente Notificador y Adaptadores (Strategy + Adapter)

En el microservicio de notificaciones combinamos con éxito los patrones Strategy y Adapter. El componente Notificador delega dinámicamente el despacho del mensaje a una interfaz EstrategiaNotificacion (Email, SMS, WhatsApp) en base al canal del usuario. Asimismo, para no esclavizar nuestro dominio a las librerías o SDKs rígidos de proveedores externos (como Twilio o Resend), introducimos contratos limpios de bajo nivel (ProveedorEmail, ProveedorSMS, ProveedorWhatsapp) implementados por adaptadores específicos (ResendAdapter, TwilioAdapter). Esto respeta el Principio de Segregación de Interfaces (ISP) y nos permite reemplazar a los proveedores de infraestructura con costo de cambio cero.

Logística - Entrega y Planificación de Rutas

Diseñamos este módulo separando rigurosamente las entidades físicas de control (Camion, Ruta, Parada, Ubicacion) de la lógica de procesamiento e integración externa, la cual fue delegada al componente de servicio GestorPlanificacionRutas. Este gestor asume la responsabilidad exclusiva de comunicarse con la API de ruteo externa, aislando la complejidad técnica de interactuar mediante callbacks (webhooks) y de fragmentar las solicitudes en lotes de máximo 100 unidades. Esta división protege la alta cohesión de la clase Ruta, permitiendo que se enfoque únicamente en el monitoreo físico de la flota y el procesamiento de geolocalizaciones en tiempo real sin interferencias de infraestructura de red.

Integracion entre servicios

Dividir el monolito inicial en tres servicios independientes responde a la necesidad de maximizar la cohesión funcional:

- **Donaciones** se enfoca exclusivamente en el estado del inventario, los donantes, las entidades y las reglas del negocio de asignación.
- **Logística** se especializa en la optimización física, el control de la flota de camiones, el ruteo y la geolocalización en tiempo real.
- **Notificaciones** se aísla como un servicio genérico de infraestructura, cuyo único propósito es interactuar con pasarelas de mensajería externas (APIs de terceros).

